NLP TUTORIAL

Traditional NLP

Text Representation · N-gram Language Models · Perplexity

Tamta Kapanadze

Supervised by Prof. Gözde Gül Şahin

What We'll Cover Today

01

Text Representation Basics

N-grams · word ⟶ index mapping

02

N-gram Language Models

Bigram counts · sentence probabilities · smoothing

03

Perplexity

Formula · MLE vs Add-1 · live comparison

04

KenLM

ARPA format · lmplz · Python API · PPL comparison

The Problem: Text → Numbers

How do we get a computer to understand text?

Computers work with **numbers**, not words.
We need to
represent text as vectors.
This is called
text representation.

S1:

"The food was not good, the service was"

vs

S2:

"The food was good, the service was not"

→ *A human instantly sees these mean opposite things. Can a computer?*

N-grams: Capturing Local Order

💡 **Idea:** Instead of single words, look at sequences of N words

S1: "the food was not good the service was"

N	Name	Examples from S1
1	Unigram	"food", "not", "good"
2	Bigram	"was not", "not good"
3	Trigram	"food was not", "was not good"

S2: "the food was good the service was not"

N	Name	Examples from S2
1	Unigram	"food", "good", "not"
2	Bigram	"was good", "was not"
3	Trigram	"food was good", "was good the"

✅ Bigrams already differentiate: ***"was not good"*** vs ***"was good"***

Our Corpus: The Little Prince

"the little prince went away he went to look at the roses
he said to the roses you are not at all like my rose
no one has tamed you and you have tamed no one
my fox was like that once"

42	28	2
Total tokens	Unique words	Model order (bigram)

→ We will build a **bigram model** from this corpus and use it to calculate sentence probabilities.

Antoine de Saint-Exupéry, 1943 · Public Domain

02

Wait — strings are inefficient!

Computers work with numbers, not strings. Comparing "the" == "the" is slow. Instead we map every word to an integer index once — then use indices everywhere.

Vocabulary → index mapping (V = 28)

```
all  →  0
and  →  1
are  →  2
at  →  3
away  →  4
fox  →  5
has  →  6
have  →  7
he  →  8
like  →  9
little  →  10
look  →  11
my  →  12
no  →  13
not  →  14
once  →  15
one  →  16
prince  →  17
rose  →  18
roses  →  19
said  →  20
tamed  →  21
that  →  22
the  →  23
to  →  24
went  →  25
was  →  26
you  →  27
```

```python
word2idx = {w: i for i, w in enumerate(vocab)}
# e.g.  word2idx["the"] = 23,  word2idx["prince"] = 17

tokens  = "the little prince went away".split()
ids     = [word2idx[w] for w in tokens]
# → [23, 10, 17, 25, 4]
```

→ From this point all count tables and probability functions use integer indices internally.

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \mathbf{Count(w_{i-1}, w_i)} \;/\; \mathbf{Count(w_{i-1})}$

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$Count(w_{i-1}, w_i)$	$Count(w_{i-1})$	$P(w_i \mid w_{i-1})$

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \mathbf{Count(w_{i-1}, w_i)} \;/\; \mathbf{Count(w_{i-1})}$

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$Count(w_{i-1}, w_i)$	$Count(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	**little**	1	3	**1/3 ≈ 0.33**

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \text{Count}(w_{i-1}, w_i) \;/\; \text{Count}(w_{i-1})$

the **little prince** went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$\text{Count}(w_{i-1}, w_i)$	$\text{Count}(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	little	1	3	$1/3 \approx 0.33$
little	**prince**	1	1	**1/1 = 1.0**

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \mathbf{Count(w_{i-1}, w_i)} \;/\; \mathbf{Count(w_{i-1})}$

the little **prince went** away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$Count(w_{i-1}, w_i)$	$Count(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	little	1	3	1/3 ≈ 0.33
little	prince	1	1	1/1 = 1.0
prince	**went**	1	1	**1/1 = 1.0**

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \text{Count}(w_{i-1}, w_i) \;/\; \text{Count}(w_{i-1})$

the little prince went away **he went** to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$\text{Count}(w_{i-1}, w_i)$	$\text{Count}(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	little	1	3	1/3 ≈ 0.33
little	prince	1	1	1/1 = 1.0
prince	went	1	1	1/1 = 1.0
he	**went**	1	2	**1/2 = 0.5**

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \mathbf{Count(w_{i-1}, w_i)} \;/\; \mathbf{Count(w_{i-1})}$

the little prince went away he went to look at the roses **he said** to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$Count(w_{i-1}, w_i)$	$Count(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	little	1	3	1/3 ≈ 0.33
little	prince	1	1	1/1 = 1.0
prince	went	1	1	1/1 = 1.0
he	went	1	2	1/2 = 0.5
he	**said**	1	2	**1/2 = 0.5**

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \mathbf{Count(w_{i-1}, w_i)} \;/\; \mathbf{Count(w_{i-1})}$

the little prince went away he went to look at **the roses** he said to **the roses** you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$Count(w_{i-1}, w_i)$	$Count(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	little	1	3	1/3 ≈ 0.33
little	prince	1	1	1/1 = 1.0
prince	went	1	1	1/1 = 1.0
he	went	1	2	1/2 = 0.5
he	said	1	2	1/2 = 0.5
the	**roses**	2	3	**2/3 ≈ 0.67**

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \mathbf{Count(w_{i-1}, w_i)} \;/\; \mathbf{Count(w_{i-1})}$

the little prince went away he went to look at the roses he **said to** the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$Count(w_{i-1}, w_i)$	$Count(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	little	1	3	1/3 ≈ 0.33
little	prince	1	1	1/1 = 1.0
prince	went	1	1	1/1 = 1.0
he	went	1	2	1/2 = 0.5
he	said	1	2	1/2 = 0.5
the	roses	2	3	2/3 ≈ 0.67
said	**to**	1	1	**1/1 = 1.0**

02

Building the Bigram Count Table

$P(w_i \mid w_{i-1}) = \mathbf{Count(w_{i-1}, w_i)} \;/\; \mathbf{Count(w_{i-1})}$

the little prince went away he went to look at the roses he said **to the** roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	$Count(w_{i-1}, w_i)$	$Count(w_{i-1})$	$P(w_i \mid w_{i-1})$
the	little	1	3	1/3 ≈ 0.33
little	prince	1	1	1/1 = 1.0
prince	went	1	1	1/1 = 1.0
he	went	1	2	1/2 = 0.5
he	said	1	2	1/2 = 0.5
the	roses	2	3	2/3 ≈ 0.67
said	to	1	1	1/1 = 1.0
to	**the**	1	2	**1/2 = 0.5**

Calculate P("he went to look")

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	**= 0.0476**

Using: **$P(w_1 w_2 \ldots w_{\square}) \approx P(w_1) \times P(w_2 | w_1) \times P(w_3 | w_2) \times \ldots$**

Calculate P("he went to look")

the little prince went away **he went** to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	= 0.0476	
P(went|he)	= 1/2	**= 0.5000**	running: 0.023810

Calculate P("he went to look")

the little prince went away he **went to** look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	= 0.0476	
P(went|he)	= 1/2	= 0.5000	*running: 0.023810*
P(to|went)	= 1/2	**= 0.5000**	running: 0.011905

Calculate P("he went to look")

the little prince went away he went **to look** at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	= 0.0476	
P(went|he)	= 1/2	= 0.5000	*running: 0.023810*
P(to|went)	= 1/2	= 0.5000	*running: 0.011905*
P(look|to)	= 1/2	**= 0.5000**	running: 0.005952

Final: P("he went to look") = 2/42 × 1/2 × 1/2 × 1/2 = **≈ 0.006** ✅

02

Your Turn: P("he said to the")

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

?

?

?

?

02

Your Turn: P("he said to the")

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	**= 0.0476**

Can you predict the steps before clicking? **Click → to reveal each one**

Your Turn: P("he said to the")

the little prince went away he went to look at the roses **he said** to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	= 0.0476	
P(said|he)	= 1/2	**= 0.5000**	running: 0.023810

Your Turn: P("he said to the")

the little prince went away he went to look at the roses he **said to** the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	= 0.0476	
P(said|he)	= 1/2	= 0.5000	*running: 0.023810*
P(to|said)	= 1/1	**= 1.0000**	running: 0.023810

Your Turn: P("he said to the")

the little prince went away he went to look at the roses he said **to the** roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

P(he)	= 2/42	= 0.0476	
P(said|he)	= 1/2	= 0.5000	*running: 0.023810*
P(to|said)	= 1/1	= 1.0000	*running: 0.023810*
P(the|to)	= 1/2	**= 0.5000**	running: 0.011905

Final: P("he said to the") = 2/42 × 1/2 × 1/1 × 1/2 = **≈ 0.012** ✅

Add-1 (Laplace) Smoothing

$$P_{Add\text{-}1}(w_i \mid w_{i-1}) = \text{Count}(w_{i-1}, w_i) + 1 \;/\; \text{Count}(w_{i-1}) + V$$

V = vocabulary size = 28

w_{i-1}	w_i	Count	MLE	Add-1
prince	went	1	1/1 = 1.0	2/29 = 0.069
he	said	1	1/2 = 0.5	2/30 = 0.067
prince	said	0	**0/1 = 0** ❌	**1/29 = 0.034** ✅
rose	was	0	**0/1 = 0** ❌	**1/29 = 0.034** ✅

✅ **Zeros are gone!** P(said|prince) goes from 0 → 0.034. No more zero probability problem.

Before vs After: Add-1 Smoothing

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	Count	MLE (before)	Add-1 (after)	Change

Before vs After: Add-1 Smoothing

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	Count	MLE (before)	Add-1 (after)	Change
the	**little**	1	0.3333	0.0645	↓ slightly

Before vs After: Add-1 Smoothing

the little prince went away **he went** to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	Count	MLE (before)	Add-1 (after)	Change
the	little	1	0.3333	0.0645	↓ slightly
he	**went**	1	0.5000	0.0667	↓ slightly

Before vs After: Add-1 Smoothing

the little prince went away he went to look at the roses **he said** to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	Count	MLE (before)	Add-1 (after)	Change
the	little	1	0.3333	0.0645	↓ slightly
he	went	1	0.5000	0.0667	↓ slightly
he	**said**	1	0.5000	0.0667	↓ slightly

Before vs After: Add-1 Smoothing

the little prince went away he went to look at **the roses** he said to **the roses** you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	Count	MLE (before)	Add-1 (after)	Change
the	little	1	0.3333	0.0645	↓ slightly
he	went	1	0.5000	0.0667	↓ slightly
he	said	1	0.5000	0.0667	↓ slightly
the	**roses**	2	0.6667	0.0968	↓ slightly

Before vs After: Add-1 Smoothing

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	Count	MLE (before)	Add-1 (after)	Change
the	little	1	0.3333	0.0645	↓ slightly
he	went	1	0.5000	0.0667	↓ slightly
he	said	1	0.5000	0.0667	↓ slightly
the	roses	2	0.6667	0.0968	↓ slightly
prince	**said**	0	**0** ❌	**0.0345** ✅	**0 → 0.0345**

Before vs After: Add-1 Smoothing

the little prince went away he went to look at the roses he said to the roses you are not at

all like my rose no one has tamed you and you have tamed no one my fox was like that once

w_{i-1}	w_i	Count	MLE (before)	Add-1 (after)	Change
the	little	1	0.3333	0.0645	↓ slightly
he	went	1	0.5000	0.0667	↓ slightly
he	said	1	0.5000	0.0667	↓ slightly
the	roses	2	0.6667	0.0968	↓ slightly
prince	said	0	0 ❌	0.0345 ✅	0 → 0.0345
rose	**was**	0	**0** ❌	**0.0345** ✅	**0 → 0.0345**

The Formula

$$PP(W) = P(w_1 w_2 \ldots w_N)^{(-1/N)}$$

$$= \sqrt[N]{1 / P(w_1 w_2 \ldots w_N)}$$

N = number of words in the test set

Lower is better

Lower perplexity = model is less surprised = better fit to the data

In practice: use log probabilities

$\log PP = -(1/N) \sum \log P(w_i | w_{i-1})$ avoids numerical underflow

Only comparable with same vocabulary

A model with V=1000 and V=50000 cannot be directly compared by perplexity

Live Example: MLE vs Add-1 Perplexity

We evaluate both models on 3 test sentences from The Little Prince.

$PP(W) = P(w_1 \ldots w_N)^{(-1/N)}$ → **lower = better** *(model is less surprised by the test data)*

Test Sentence	Words	P_MLE	P_Add1	Running PPL (MLE)	Running PPL (Add-1)

03

Live Example: MLE vs Add-1 Perplexity

We evaluate both models on 3 test sentences from The Little Prince. *Click → to reveal each sentence.*

$PP(W) = P(w_1 \ldots w_N)^{(-1/N)}$ → **lower = better** *(model is less surprised by the test data)*

Test Sentence	Words	P_MLE	P_Add1	Running PPL (MLE)	Running PPL (Add-1)
"he went to look at the roses"	7	0.001984	0.00000001	**2.94**	**100.00**

03

Live Example: MLE vs Add-1 Perplexity

We evaluate both models on 3 test sentences from The Little Prince. *Click → to reveal each sentence.*

PP(W) = P(w_1...w□)^(−1/N) → **lower = better** *(model is less surprised by the test data)*

Test Sentence	Words	P_MLE	P_Add1	Running PPL (MLE)	Running PPL (Add-1)
"he went to look at the roses"	7	0.001984	0.00000001	2.94	100.00
"the little prince went away"	5	0.011905	0.00000146	**2.68**	**60.30**

03

Live Example: MLE vs Add-1 Perplexity

We evaluate both models on 3 test sentences from The Little Prince. *Click → to reveal each sentence.*

$PP(W) = P(w_1 \ldots w_N)^{(-1/N)}$ → **lower = better** *(model is less surprised by the test data)*

Test Sentence	Words	P_MLE	P_Add1	Running PPL (MLE)	Running PPL (Add-1)
"he went to look at the roses"	7	0.001984	0.00000001	2.94	100.00
"the little prince went away"	5	0.011905	0.00000146	2.68	60.30
"he said to the roses you are not"	8	0.001323	0.00000000	**2.37**	**14.82**

MLE PPL = 2.37 ✅ vs **Add-1 PPL = 14.82** ⚠️

MLE wins here — but only because these sentences are all from the training corpus!

04

Section 4: KenLM — Industrial-Strength LMs

Our implementation was built for learning. In practice, the field uses optimised toolkits that handle billion-token corpora and state-of-the-art smoothing.

KenLM (2011)

Key advantage

Binary format

Our implementation		KenLM
Python dicts for counts	→	Memory-mapped binary indexes
Add-1 / Add-k smoothing	→	Modified Kneser-Ney (state of the art)
Sentence-level PPL	→	Corpus-level PPL in one command
Handles tiny corpus (42 tokens)	→	Handles billions of tokens

Section 4: KenLM — Industrial-Strength LMs

The ARPA format — standard LM representation

```
\data\
ngram 1=28      # unique unigrams
ngram 2=41      # unique bigrams

\1-grams:
-1.255   the    0.000     # log10 P(the)  [tab]  word  [tab]  backoff
-1.556   away   0.000

\2-grams:
-1.176   the little       # log10 P(little | the)
-1.176   little prince

\end\
```

Training with lmplz

```
# Train a bigram model from raw text
lmplz -o 2 --discount_fallback < corpus.txt > model.arpa

# Convert to fast binary format (~10x faster load)
build_binary model.arpa model.bin
```

Section 4: KenLM — Industrial-Strength LMs

Querying KenLM from Python

```python
import kenlm

model = kenlm.Model("model.bin")

# log10 probability of the full sentence
model.score("he went to look at the roses")
# → (KN log10 prob)

# perplexity normalised by token count
model.perplexity("he went to look at the roses")
# → 3.59

# per-token breakdown
for log10p, ngram_len, oov in model.full_scores(s):
    print(log10p, ngram_len, oov)
```

`full_scores("he went to look")`

Word	log10 P	order	OOV
he	-0.55	2	no
went	-0.48	2	no
to	-0.48	2	no
look	-0.48	2	no
</s>	-0.72	1	no

order=2 → full bigram was used; order=1 → model backed off to unigram

Section 4: KenLM — Industrial-Strength LMs

KenLM (Kneser-Ney) vs our Add-1 — side by side

Sentence	Ours (Add-1)	KenLM (KN)
"he went to look at the roses"	14.85	3.59
"the little prince went away"	14.69	4.99
"he said to the roses you are not"	14.87	4.40

Why different? KenLM uses Kneser-Ney smoothing — KN discounts observed bigrams and redistributes mass based on word versatility across contexts. This gives much lower PPL than Add-1's flat +1 pseudocount.

Mini exercise — try in the notebook (§4.6)

```
unseen = ["the prince looked at the rose",  # "looked" is OOV
          "the little fox went away"]       # all words known

for s in unseen:
    print(model.perplexity(s))   # KN backs off to smarter unigrams; OOV handled via <unk>
```